PRODUCT REQUIREMENTS DOCUMENT (PRD)

Hospital Management System REST API

Technical Specification, Domain Invariants & Implementation Guidelines for Engineering Teams

ARCHITECTURE STYLE	**RUNTIME / FRAMEWORK**
Backend-Only REST API (Headless)	Java 17 LTS / Spring Boot 3.3.4
SERVER PORT & BASE URL	**SECURITY & AUTHENTICATION**
8085 (http://localhost:8085/)	Stateless JWT / Bearer Token
PRIMARY DATABASE	**INTERACTIVE DOCUMENTATION**
MySQL 8.x (H2 In-Memory Fallback)	Swagger UI (/swagger-ui/index.html)

Product Requirements Document (PRD)

Hospital Management System REST API

1. Project Overview

The **Hospital Management System (HMS) REST API** is an enterprise-grade, backend-only healthcare information management platform built on **Java 17, Spring Boot 3.3.4, Spring Data JPA, Hibernate, Spring Security, JWT, and MySQL/H2**.

The system provides a unified, headless service architecture that coordinates clinical encounters, patient records, provider schedules, pharmacy logistics, inpatient ward and bed occupancy, diagnostic orders, itemized billing, and executive operational analytics. All user interactions are mediated strictly via RESTful HTTP interfaces authenticated with JSON Web Tokens (JWT), with interactive exploration and developer testing enabled via **Springdoc OpenAPI (Swagger UI)** at `http://localhost:8085/swagger-ui/index.html`.

The primary goal of this platform is to establish transactional consistency, eliminate double-booking and stock overselling anomalies, enforce clinical triage prioritization, and provide an extensible API foundation for external web portals, mobile apps, and hospital IoT integrations.

2. Feature List

1. **Authentication & Identity Access Management (IAM)**: Provides secure stateless JWT registration, login, token parsing, and role-based authorization for administrators, doctors, and patients.
2. **Patient Demographic & Medical Record Management**: Stores comprehensive patient profiles and aggregates a unified historical timeline of all clinical encounters, vitals, prescriptions, admissions, and bills.
3. **Doctor & Clinical Staff Directory**: Manages doctor credentials, departmental affiliations, medical license verification, consultation fees, and real-time availability states.
4. **Hospital Department Structure**: Organizes clinical specialties, administrative units, and departmental service routing.
5. **Walk-In Outpatient Queue Management**: Handles unscheduled patient arrivals with atomic, daily-sequenced tokens (`W-001` , `W-002` , ...) and active queue transitions.
6. **Emergency Bay Clinical Triage**: Prioritizes urgent trauma admissions based on physiological acuity levels (`CRITICAL` > `HIGH` > `MEDIUM` > `LOW`) ensuring immediate clinical intervention.
7. **Doctor Appointment Scheduling**: Coordinates bookable consultation slots while enforcing an invariant that prevents doctor schedule overlapping and double-booking collisions.
8. **Clinical Consultation & Diagnostics**: Records patient symptoms, doctor diagnoses, physiological vital signs, and follow-up consultation dates.
9. **Digital Prescription Issuance**: Allows physicians to issue itemized medication orders linked directly to consultations with explicit dosages, frequencies, and durations.
10. **Pharmacy Inventory Control**: Tracks medicine batches, supplier invoices, minimum stock reorder thresholds, and shelf expiration dates.
11. **Point-of-Sale (POS) Pharmacy Dispensing**: Dispenses prescription and over-the-counter medications with atomic stock deduction, expired drug rejection, and financial total calculation.

12. **Inpatient Ward & Bed Lifecycle Management**: Allocates available hospital beds across specialized wards (General, ICU, Emergency, Private) with automatic status locking on admission and release on discharge.
13. **Inpatient Admission & Discharge (ADT)**: Manages inpatient hospital stays from admission diagnosis to clinical discharge summaries.
14. **Itemized Billing & Payment Processing**: Aggregates consultation fees, room/bed tariffs, medication charges, and diagnostic fees into a unified invoice with multi-method payment settlement.
15. **Departmental Inquiries & Patient Messaging**: Routes patient inquiries and feedback to specific hospital departments with read-state tracking.
16. **Executive Dashboard & Analytical Reporting**: Delivers real-time operational KPI summaries (bed occupancy, patient influx, revenue summaries, and low-stock alerts).

---

3. Functional Requirements

3.1 Authentication & User Management

- **User Registration**:
- Accepts `firstName`, `lastName`, `email`, `password`, `phone`, `dateOfBirth`, `gender`, and optional clinical fields (`bloodGroup`, `emergencyContact`).
- Passwords must be encoded using BCrypt (`BCryptPasswordEncoder`).
- Rejects registrations if the email address already exists (`DuplicateResourceException`).
- **User Login**:
- Validates credentials against stored BCrypt hashes.
- On success, issues a signed HMAC-SHA256 JWT containing `sub` (email), `userId`, and `role` claims, valid for 24 hours (configurable via `JWT_EXPIRATION_MS`).
- **Authorization Roles**:
- `ADMIN`: Unrestricted access across all domains, inventory purchases, doctor registrations, and reports.
- `DOCTOR`: Clinical access (consultations, prescriptions, triage, admissions, appointments).
- `PATIENT`: Self-service access (own appointments, own medical history, own prescriptions/invoices, messaging).

3.2 Walk-In Outpatient Queue Flow

- **Token Generation**:
- Daily tokens must be generated sequentially in format `W-001`, `W-002`, `W-003`, ..., resetting each day at midnight (UTC/local).
- Computed atomically by counting the current calendar day's registered walk-ins (`WalkInVisitRepository.countByVisitDate(today) + 1`).
- **Lifecycle States**:
- `WAITING` → Initial state upon front-desk check-in.
- `CALLED` → Attending nurse or physician calls patient into consultation room.
- `IN_CONSULTATION` → Active clinical encounter.
- `COMPLETED` → Consultation concluded.
- `CANCELLED` → Patient left before being called.
- **Edge Cases**:
- Cancelling or deleting a walk-in record must not retroactively renumber existing tokens.

3.3 Emergency Bay Clinical Triage

- **Acuity Classification**:
- `CRITICAL` : Immediate life threat (cardiac arrest, severe trauma, respiratory failure). Requires resuscitation.
- `HIGH` : Emergent conditions (< 15-minute intervention target).
- `MEDIUM` : Urgent conditions (< 30-minute intervention target).
- `LOW` : Non-urgent emergency bay arrivals.
- **Queue Priority Sorting**:
- The active emergency queue (`GET /api/emergency/active`) must strictly sort patients with `CRITICAL` priority first, followed by `HIGH` , `MEDIUM` , and `LOW` , sub-ordered by `arrivalTime ASC` .
- **Status Lifecycle**:
- `REGISTERED` → `TRIAGE` → `UNDER_TREATMENT` → `ADMITTED` (transfers to inpatient bed) or `DISCHARGED` .

3.4 Doctor Appointment Scheduling & Collision Prevention

- **Double-Booking Invariant**:
- When booking an appointment (`POST /api/appointments`), the system must verify:

```
SELECT COUNT(*) FROM appointments
WHERE doctor_id = :doctorId
  AND appointment_date = :date
  AND appointment_time = :time
  AND status NOT IN ('CANCELLED', 'REJECTED')
```

- If a record exists, the transaction must fail immediately with `DoubleBookingException` (HTTP 409 Conflict).
- **Patient Self-Booking**:
- Authenticated patients can only create appointments for themselves.
- Administrators can schedule on behalf of any patient.

3.5 Pharmacy Inventory & Medication Dispensing

- **Dispensing Invariants**:
- **Expiry Check**: For each item in a sale request, if `medicine.expiryDate.isBefore(LocalDate.now())` , reject the entire transaction with `ExpiredMedicineException` (HTTP 400 Bad Request).
- **Stock Check**: If `medicine.quantity < requestedQuantity` , reject the transaction with `InsufficientStockException` (HTTP 400 Bad Request).
- **Atomic Stock Deduction**:
- Inside a `@Transactional` boundary, decrement `medicine.quantity` by the sold quantity.
- Auto-calculate subtotal, tax (e.g. 5%), discount, and net total.
- **Inventory Monitoring**:
- Trigger warning alerts when `medicine.quantity <= medicine.minimumStockLevel` .

3.6 Inpatient Ward & Bed Allocation

- **Bed Locking**:
- When admitting a patient (`POST /api/admissions`), target bed must have status `AVAILABLE` .
- If target bed status is `OCCUPIED` , `RESERVED` , or `MAINTENANCE` , reject with `BedUnavailableException` (HTTP 400).
- Atomically update bed status to `OCCUPIED` .

- **Bed Release**:
- When patient is discharged (`PUT /api/admissions/{id}/discharge`), set admission status to `DISCHARGED` , stamp `dischargeDate` , and transition bed status back to `AVAILABLE` .

3.7 Invoicing & Settlement

- **Aggregation**:
- Invoices calculate total line items across Consultation Fees, Bed/Room Charges, Dispensed Medications, and Diagnostic Procedures.
- Subtotal, discounts, applicable taxes, and final payable balance must use `BigDecimal` for zero rounding errors.
- **Payments**:
- Recorded against invoice ID with payment method (`CASH` , `CARD` , `UPI` , `INSURANCE`) and reference ID.

---

4. Data Structures

The system data domain is modeled in Java persistence entities (`jakarta.persistence`) mapped to relational database schemas. Below are the corresponding structural definitions (represented in TypeScript interface notation for client-side API consumption).

```
// Core Identity & User
export enum Role {
  ADMIN = "ADMIN",
  DOCTOR = "DOCTOR",
  PATIENT = "PATIENT"
}

export interface User {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
  phone: string;
  role: Role;
  aadhar?: string;
  gender?: string;
  dateOfBirth?: string; // YYYY-MM-DD
  createdAt: string;    // ISO-8601
  updatedAt: string;
}

// Patient Record
export interface Patient {
  id: number;
  patientId: string; // e.g. PAT-2026-00001
  userId?: number;
  firstName: string;
  lastName: string;
  email: string;
  phone: string;
  dateOfBirth: string;
  gender: string;
  bloodGroup: string;
  emergencyContact: string;
  address?: string;
  allergies?: string;
  medicalHistory?: string;
  createdAt: string;
}

// Doctor & Provider
export enum DoctorStatus {
  AVAILABLE = "AVAILABLE",
  BUSY = "BUSY",
  ON_LEAVE = "ON_LEAVE",
  INACTIVE = "INACTIVE"
}

export interface Doctor {
  id: number;
  userId?: number;
  doctorId: string; // e.g. DOC-2026-0001
  firstName: string;
  lastName: string;
  email: string;
  phone: string;
  department: string;
  departmentId?: number;
  qualification: string;
  medicalLicense: string;
  experienceYears: number;
  consultationFee: number;
  status: DoctorStatus;
}
```

```
// Walk-In Outpatient
export enum WalkInStatus {
  WAITING = "WAITING",
  CALLED = "CALLED",
  IN_CONSULTATION = "IN_CONSULTATION",
  COMPLETED = "COMPLETED",
  CANCELLED = "CANCELLED"
}

export interface WalkInVisit {
  id: number;
  tokenNumber: string; // e.g. W-001
  patientId: number;
  patientName?: string;
  departmentId: number;
  departmentName?: string;
  doctorId?: number;
  doctorName?: string;
  visitReason: string;
  priority: "LOW" | "MEDIUM" | "HIGH";
  status: WalkInStatus;
  arrivalTime: string;
  consultationStartTime?: string;
  consultationEndTime?: string;
}

// Emergency Bay
export enum EmergencyPriority {
  CRITICAL = "CRITICAL",
  HIGH = "HIGH",
  MEDIUM = "MEDIUM",
  LOW = "LOW"
}

export enum EmergencyStatus {
  REGISTERED = "REGISTERED",
  TRIAGE = "TRIAGE",
  UNDER_TREATMENT = "UNDER_TREATMENT",
  ADMITTED = "ADMITTED",
  DISCHARGED = "DISCHARGED"
}

export interface EmergencyCase {
  id: number;
  patientId: number;
  chiefComplaint: string;
  emergencyCondition: string;
  priority: EmergencyPriority;
  status: EmergencyStatus;
  assignedDoctorId?: number;
  departmentId?: number;
  bloodPressure?: string;
  heartRate?: number;
  oxygenSaturation?: number;
  temperature?: number;
  respiratoryRate?: number;
  treatmentNotes?: string;
  arrivalTime: string;
}

// Appointments
export enum AppointmentStatus {
  PENDING = "PENDING",
  CONFIRMED = "CONFIRMED",
  CHECKED_IN = "CHECKED_IN",
  IN_CONSULTATION = "IN_CONSULTATION",
```

```
  COMPLETED = "COMPLETED",
  CANCELLED = "CANCELLED"
}

export interface Appointment {
  id: number;
  patientId: number;
  doctorId: number;
  departmentId?: number;
  appointmentDate: string; // YYYY-MM-DD
  appointmentTime: string; // HH:mm:ss
  reason: string;
  status: AppointmentStatus;
  notes?: string;
}

// Pharmacy & Inventory
export interface Medicine {
  id: number;
  medicineName: string;
  category: string;
  manufacturer: string;
  batchNumber: string;
  expiryDate: string; // YYYY-MM-DD
  purchasePrice: number;
  sellingPrice: number;
  quantity: number;
  minimumStockLevel: number;
  prescriptionRequired: boolean;
}

export interface PharmacySaleItem {
  id?: number;
  medicineId: number;
  medicineName?: string;
  quantity: number;
  unitPrice: number;
  totalPrice: number;
}

export interface PharmacySale {
  id: number;
  patientId?: number;
  prescriptionId?: number;
  subtotal: number;
  tax: number;
  discount: number;
  totalAmount: number;
  paymentMethod: "CASH" | "CARD" | "UPI" | "INSURANCE";
  saleDate: string;
  items: PharmacySaleItem[];
}

// Wards & Beds
export enum BedStatus {
  AVAILABLE = "AVAILABLE",
  OCCUPIED = "OCCUPIED",
  RESERVED = "RESERVED",
  MAINTENANCE = "MAINTENANCE"
}

export interface Bed {
  id: number;
  wardId: number;
  bedNumber: string;
  status: BedStatus;
```

```
}

export interface Ward {
  id: number;
  name: string;
  wardType: "GENERAL" | "ICU" | "EMERGENCY" | "PRIVATE";
  floorNumber: number;
  totalBeds: number;
}

// Inpatient Admission
export enum AdmissionStatus {
  ADMITTED = "ADMITTED",
  UNDER_TREATMENT = "UNDER_TREATMENT",
  READY_FOR_DISCHARGE = "READY_FOR_DISCHARGE",
  DISCHARGED = "DISCHARGED"
}

export interface Admission {
  id: number;
  patientId: number;
  doctorId: number;
  departmentId: number;
  wardId: number;
  bedId: number;
  admissionDate: string;
  dischargeDate?: string;
  reason: string;
  diagnosis?: string;
  status: AdmissionStatus;
}
```

---

5. API Interfaces

The system is configured with base path `/` and server port `8085`. Protected endpoints require HTTP Header `Authorization: Bearer`.

5.1 Authentication API (`/api/auth/**`)

Endpoint	Method	Role	Request Body	Response Body
`/api/auth/register`	**POST**	Public	`RegisterRequest`	`201 Created` (`AuthResponse`)
`/api/auth/login`	**POST**	Public	`LoginRequest`	`200 OK` (`AuthResponse`)
`/api/auth/me`	**GET**	Authenticated	None	`200 OK` (`UserResponse`)

Usage Example: User Authentication

```
POST /api/auth/login HTTP/1.1
Host: localhost:8085
Content-Type: application/json

{
  "email": "admin@hms.com",
  "password": "Admin@123"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "User Login Successfully",
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbkBobXMuY29tIiwiZXhwIjoxNzk...",
  "user": {
    "id": 1,
    "firstName": "System",
    "lastName": "Admin",
    "email": "admin@hms.com",
    "role": "ADMIN"
  }
}
```

5.2 Patient API (`/api/patients/**`)

Endpoint	Method	Role	Description
`/api/patients`	`GET`	Admin / Doctor	List all registered patients
`/api/patients/{id}`	`GET`	Admin / Doctor / Patient (Own)	Retrieve patient details by ID
`/api/patients`	`POST`	Admin	Create patient record manually
`/api/patients/{id}`	`PUT`	Admin	Update patient medical record
`/api/patients/{id}/medical-history`	`GET`	Admin / Doctor / Patient (Own)	Unified patient timeline summary

5.3 Walk-In Queue API (`/api/walk-ins/**`)

Endpoint	Method	Role	Description
`/api/walk-ins`	`POST`	Patient / Admin	Register walk-in arrival; returns token `W-001`
`/api/walk-ins/today`	`GET`	Admin / Doctor	List all walk-in visits for today
`/api/walk-ins/department/{deptId}/queue`	`GET`	Admin / Doctor	Active queue filtered by department
`/api/walk-ins/{id}/status`	`PUT`	Admin / Doctor	Update visit state (`WAITING` → `CALLED` → `COMPLETED`)

5.4 Emergency Department API (`/api/emergency/**`)

Endpoint	Method	Role	Description
`/api/emergency`	`POST`	Admin / Doctor	Register emergency patient with vital signs
`/api/emergency/active`	`GET`	Admin / Doctor	Active emergency queue sorted: `CRITICAL` first
`/api/emergency/{id}/triage`	`PUT`	Doctor	Adjust clinical triage priority level
`/api/emergency/{id}/status`	`PUT`	Doctor	Progress status (`TRIAGE` , `ADMITTED` , `DISCHARGED`)

5.5 Appointments API (`/api/appointments/**`)

Endpoint	Method	Role	Description
`/api/appointments`	`POST`	Patient / Admin	Book appointment (intercepts double-booking)
`/api/appointments/doctor/{doctorId}`	`GET`	Doctor / Admin	Retrieve doctor appointment schedule
`/api/appointments/patient/{patientId}`	`GET`	Patient / Admin	Retrieve patient scheduled appointments
`/api/appointments/{id}/status`	`PUT`	Doctor / Admin	Confirm, check in, or complete appointment
`/api/appointments/{id}`	`DELETE`	Patient / Admin	Cancel appointment

5.6 Pharmacy & Inventory API (`/api/inventory/` & `/api/pharmacy/`)

Endpoint	Method	Role	Description
`/api/inventory`	`GET`	Admin / Doctor	Full inventory catalog with batch numbers
`/api/inventory`	`POST`	Admin	Add new medicine batch with expiry and price
`/api/inventory/low-stock`	`GET`	Admin	Medicines with stock ≤ minimumStockLevel
`/api/inventory/expired`	`GET`	Admin	Expired medicine inventory
`/api/pharmacy/sales`	`POST`	Admin	Dispense medication; validates expiry and stock

5.7 Inpatient Beds & Admissions API (`/api/beds/` & `/api/admissions/`)

Endpoint	Method	Role	Description
`/api/wards`	`GET`	Admin / Doctor	Hospital wards (General, ICU, Emergency, Private)
`/api/beds/available`	`GET`	Admin / Doctor	Unoccupied beds ready for patient allocation
`/api/admissions`	`POST`	Doctor / Admin	Admit patient to bed; locks bed to `OCCUPIED`
`/api/admissions/active`	`GET`	Admin / Doctor	Currently admitted inpatient list
`/api/admissions/{id}/discharge`	`PUT`	Doctor / Admin	Discharge patient; releases bed to `AVAILABLE`

5.8 Invoicing & Billing API (`/api/billing/**`)

Endpoint	Method	Role	Description
`/api/billing/invoices`	`POST`	Admin	Create itemized invoice with service charges
`/api/billing/invoices/{id}`	`GET`	Admin / Patient	View invoice breakdown and balance
`/api/billing/invoices/{id}/payments`	`POST`	Admin	Settle payment (Cash, Card, UPI, Insurance)

5.9 Dashboard & Reporting API (`/api/dashboard/` & `/api/reports/`)

Endpoint	Method	Role	Description
`/api/dashboard/summary`	`GET`	Admin	High-level metrics: beds, patients, doctors, alerts
`/api/reports/revenue`	`GET`	Admin	Revenue aggregates filtered by start/end date
`/api/reports/bed-occupancy`	`GET`	Admin	Ward-level bed occupancy percentages

---

6. User Interface (UI) Outline

Note:
Backend-Only Architecture Note:

This project operates strictly as a **Headless RESTful API**. All frontend codebases (React/Vite/HTML/CSS) have been removed. The sole developer and operational user interface provided by the application is the **Springdoc OpenAPI / Swagger UI**.

6.1 Swagger UI Portal (`http://localhost:8085/swagger-ui/index.html`)

The Swagger UI provides an interactive web interface partitioned into **16 Grouped OpenAPI Modules**:

1. `0. All APIs` — Global API catalog.
2. `1. Authentication` — User registration, login, and `/me` endpoints.
3. `2. Patients` — Patient demographic CRUD and timeline aggregation.
4. `3. Doctors` — Provider profiles, specialization, and availability toggles.
5. `4. Departments` — Clinical departments.
6. `5. Walk-In Outpatient` — Token issuing (`W-001`) and queue management.
7. `6. Emergency Bay` — Clinical triage sorting and vital monitoring.
8. `7. Appointments` — Slot scheduling with double-booking rejection.
9. `8. Consultations` — Physician diagnostic recording.
10. `9. Prescriptions` — Medication regimens.
11. `10. Pharmacy & Inventory` — Stock levels, expiry alerts, and POS dispensing.
12. `11. Inpatient Beds` — Ward and bed statuses (`AVAILABLE` , `OCCUPIED`).
13. `12. Inpatient Admissions` — Patient admission and discharge workflows.
14. `13. Billing & Invoicing` — Invoices and payment collection.
15. `14. Department Messaging` — Inquiries and read-state management.
16. `15. Executive Dashboard & Reports` — Analytics and KPI metrics.

6.2 UI Consumer Interface Guidelines (For Frontend Developers)

Developers implementing web or mobile frontends against this API should structure screens as follows:

- **Admin Dashboard**: Top summary metric cards (Available Beds, Today's Appointments, Low Stock Alerts, Emergency Count) pulling from `GET /api/dashboard/summary` .
- **Triage Nurse Console**: Real-time table pulling from `GET /api/emergency/active` , displaying color-coded badges for `CRITICAL` (Red), `HIGH` (Orange), `MEDIUM` (Yellow), and `LOW` (Green).
- **Outpatient Reception Kiosk**: Simple form invoking `POST /api/walk-ins` , printing or displaying the generated `tokenNumber` (e.g. `W-001`).
- **Doctor Consultation Workspace**: Side-by-side view with patient timeline from `GET /api/patients/{id}/medical-history` , appointment details, and diagnosis/prescription submission forms.

---

7. Non-Functional Requirements

7.1 Performance & Concurrency

- **Non-Blocking Throughput**: Configured with embedded Apache Tomcat thread pool supporting concurrent clinical requests.
- **Connection Pooling**: HikariCP connection pool configured for rapid database checkout and automatic statement reuse.
- **Atomic Operations**: Walk-in token sequences and bed occupancy checks execute in isolated `@Transactional` blocks to prevent race conditions during high patient arrival volume.

7.2 Security & Compliance

- **Stateless Authentication**: Zero server-side session state; all requests authenticated via cryptographic JWT signatures.
- **Cryptographic Hashing**: User passwords hashed with standard BCrypt work factor (cost 10).
- **CORS Configuration**: Allowed origins, headers, and methods configurable in `SecurityConfig.java` to prevent cross-site request forgery when consumed by web applications.
- **Least-Privilege RBAC**: Method-level and URL-level security using Spring Security annotations (`@PreAuthorize` , `requestMatchers`).

7.3 Reliability & Availability

- **Actuator Health Monitoring**: Health endpoint exposed at `http://localhost:8085/actuator/health` returns `{"status":"UP"}` for uptime probes and load balancers.
- **Dual Persistence Profiles**:
- `mysql` : Production profile utilizing MySQL 8.x with connection retries.
- `h2` : Dev/testing profile providing in-memory database with MySQL syntax compatibility mode (`MODE=MySQL`).

7.4 Maintainability & Clean Code

- **Layered Architecture**: Strict separation of concerns across Controller, Service, Repository, Entity, and DTO layers.
- **Lombok Integration**: Reduces boilerplate for getters, setters, builders, and constructors.
- **Centralized Exception Handling**: `GlobalExceptionHandler` intercepts domain exceptions (`DoubleBookingException` , `ExpiredMedicineException` , `BedUnavailableException` , `ResourceNotFoundException`) and returns standardized `ApiResponse` objects with appropriate HTTP error codes.

---

8. Assumptions and Uncertainties

8.1 Assumptions

1. **Currency**: All monetary fields (`consultationFee` , `purchasePrice` , `sellingPrice` , `totalAmount`) are assumed to be in standard decimal currency units (e.g., USD or INR) represented with `BigDecimal` with 2 decimal precision.
2. **Timezone**: Date and time values are calculated based on server system clock (`LocalDate.now()` , `LocalDateTime.now()`). In distributed multi-region deployments, UTC standardization is assumed.
3. **Daily Token Reset**: The walk-in daily token sequence assumes the date of arrival determines the sequence index (`countByVisitDate(today) + 1`).

8.2 Uncertainties & Items Flagged for Review

- **Automatic Insurance Claims Integration**: The payment model supports `PaymentMethod.INSURANCE` and records a transaction reference, but third-party clearinghouse EDI claim filing (HL7/FHIR) is not currently implemented in code.
- **SMS / Email Gateway**: The messaging and appointment notification workflows store records in the database, but external SMS/Email dispatching adapters (e.g., Twilio, SendGrid) are stubs or absent in the existing source tree.
- **Doctor Shift Scheduling**: Doctor availability is managed via a manual status flag (`AVAILABLE` , `BUSY` , `ON_LEAVE` , `INACTIVE`) rather than an automated recurrent weekly shift roster calendar.